

Sprint Backlog — Plateforme QCM Interactive (Sujet 21 — type Kahoot!)

Stack technique

- Backend : Java Spring Boot, Spring Data JPA, Spring Security, Flyway (migrations)
 - Frontend : React (+ Tailwind ou équivalent), Axios
 - Base de données : PostgreSQL
 - Temps réel : WebSocket (STOMP over SockJS) pour les sessions live
 - Module IA : service dédié (extraction de contenu PDF/Word/PPT + génération de questions)
 - Méthodologie : Scrum, sprints de 2 semaines (Sprint 0 : 1 semaine de setup)
-

Vue d'ensemble du Product Backlog

Module

-
- | | |
|---|---|
| 1 | Authentification & Gestion des utilisateurs (Utilisateur, Role, Enseignant, Etudiant, Administrateur) |
| 2 | Structure scolaire (Classe, Matière) |
| 3 | Création de QCM (QCM, Question, Reponse) |
| 4 | Sessions live & Participations (SessionQuiz, Participation, ReponseEtudiant, StatistiqueQuestion) |
| 5 | Gamification, Notifications, Historique, Export |
| 6 | Module IA — génération de QCM à partir de documents |
| 7 | Finalisation, sécurité, optimisation, déploiement |
-

SPRINT 0 — Setup & Architecture (1 semaine)

1. Objectif du sprint

Mettre en place les fondations techniques du projet : structure des dépôts, environnements, CI/CD basique, squelette Spring Boot et squelette React, connexion à PostgreSQL, et premier déploiement "Hello World" fonctionnel de bout en bout.

2. User Stories

- En tant que développeur, je veux un projet Spring Boot initialisé, afin de pouvoir développer les modules métier dessus.
- En tant que développeur, je veux un projet React initialisé, afin de pouvoir développer l'interface utilisateur.

- En tant que développeur, je veux une base PostgreSQL connectée via Flyway, afin de versionner le schéma dès le départ.

3. Tâches détaillées

Backend

- initialiser le projet Spring Boot (Web, Data JPA, Security, Validation, WebSocket)
- configurer `application.yml` (profils dev/test/prod)
- configurer Flyway (dossier `db/migration`)
- mettre en place la structure en couches : `controller / service / repository / dto / entity / exception / config`
- configurer CORS pour le frontend
- configurer un `GlobalExceptionHandler`
- configurer Swagger/OpenAPI

Frontend

- initialiser le projet React (Vite)
- configurer le routing (React Router)
- configurer Axios (instance avec intercepteurs)
- mettre en place la structure des dossiers (`pages / components / services / hooks / context`)
- configurer Tailwind (ou équivalent) et la charte graphique de base

Base de données

- créer la base PostgreSQL locale et de test
- première migration Flyway `V1__init.sql` (schéma vide, extensions si besoin)

DevOps

- initialiser le dépôt Git (branche `main` + `develop`, convention Git Flow)
- configurer un pipeline CI basique (build + tests) — GitHub Actions
- Dockerfile backend + frontend (facultatif si déploiement prévu)

4. Diagrammes UML à modifier

- Diagramme de déploiement : poser l'architecture cible (client React, serveur Spring Boot, base PostgreSQL, éventuel service IA séparé)
- Diagramme de composants : définir les grands modules (Auth, Structure Scolaire, QCM, Sessions, Gamification, IA) et leurs dépendances

5. Base de données

- Aucune table métier ce sprint — uniquement l'initialisation du schéma Flyway et la table technique `flyway_schema_history` (auto-générée)

6. API à développer

- `GET /health` → vérifie que le backend répond (200 OK)

7. Interface utilisateur

- Page de test "Hello World" affichant un appel réussi à `/health`
- Layout de base (header vide, container principal)

8. Tests

- Test unitaire : contexte Spring démarre correctement
- Test d'intégration : connexion à la base de test réussie
- Test API : `GET /health` retourne 200

9. Critères d'acceptation

- Le backend démarre sans erreur et se connecte à PostgreSQL
- Le frontend démarre et affiche la page de test
- Le pipeline CI passe au vert

10. Dépendances

- Aucune dépendance (premier sprint)
- Risque : mauvais choix d'architecture initiale difficile à corriger plus tard → valider l'architecture avec l'encadrant avant de continuer

11. Livrables

- Squelette backend + frontend fonctionnels et connectés
- Pipeline CI opérationnel
- Diagramme de déploiement et de composants v1

12. Démonstration du sprint

- Lancer le backend et le frontend, montrer l'appel réussi à `/health` depuis l'interface

13. Check-list de validation

- ☐ Backend Spring Boot démarre
- ☐ Frontend React démarre
- ☐ Base PostgreSQL connectée via Flyway
- ☐ CI/CD basique fonctionnel
- ☐ Structure de dossiers backend/frontend en place
- ☐ Git Flow initialisé
- ☐ Diagrammes de déploiement/composants créés

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Setup Spring Boot	Facile	0.5 j	Haute
Setup React + Tailwind	Facile	0.5 j	Haute
Config Flyway + PostgreSQL	Moyen	0.5 j	Haute
CI/CD basique	Moyen	1 j	Moyenne
Diagrammes déploiement/composants	Facile	0.5 j	Moyenne

15. Bonnes pratiques

- Architecture en couches claire dès le départ (controller/service/repository)
- Convention de nommage cohérente (camelCase Java, kebab-case pour les routes)
- `.env`/`application-*.yaml` jamais commités avec des secrets
- Git Flow : `feature/*`, `develop`, `main`
- README à jour dès le premier commit

SPRINT 1 — Authentification & Gestion des Utilisateurs (2 semaines)

1. Objectif du sprint

Permettre l'inscription/connexion sécurisée des utilisateurs, la gestion des rôles (Admin, Enseignant, Étudiant), et les opérations CRUD de base sur les profils.

2. User Stories

- En tant qu'administrateur, je veux créer des comptes Enseignant et Étudiant, afin de gérer les accès à la plateforme.
- En tant qu'utilisateur, je veux me connecter avec mon email/mot de passe, afin d'accéder à mon espace personnel selon mon rôle.
- En tant qu'utilisateur, je veux modifier mon profil (nom, photo, téléphone), afin de garder mes informations à jour.
- En tant qu'administrateur, je veux désactiver un compte, afin de bloquer l'accès d'un utilisateur sans le supprimer.

3. Tâches détaillées

Backend

- créer le modèle `Utilisateur` (abstrait), `Role`, `Enseignant`, `Etudiant`, `Administrateur` (héritage JOINED ou SINGLE_TABLE selon choix)
- créer les DTOs (RegisterDTO, LoginDTO, UserResponseDTO, UpdateProfileDTO)
- créer les validations (email valide, mot de passe fort, matricule unique)

- créer le service d'authentification (hash bcrypt, génération JWT)
- créer `UserDetailsService` custom pour Spring Security
- configurer Spring Security (filtres JWT, règles d'autorisation par rôle)
- créer les routes Auth et Utilisateur
- gérer les erreurs (email déjà utilisé, identifiants invalides, compte désactivé)
- écrire les tests unitaires et d'intégration
- documenter l'API (Swagger)

Frontend

- créer la page de connexion
- créer la page d'inscription (selon rôle si autorisé)
- créer la page "Mon Profil" (édition)
- créer le formulaire de création de compte par l'admin
- connecter l'API (login, register, update profile)
- gérer les erreurs (messages clairs sous les champs)
- gérer le loading (spinners sur les boutons de soumission)
- responsive design (mobile-first pour la page de connexion)
- mettre en place le `AuthContext` (token, rôle, utilisateur courant) + routes protégées

Base de données

- créer les tables `utilisateur`, `role`, `enseignant`, `etudiant`, `administrateur`
- relations: `utilisateur.role_id → role.id`
- contraintes: `email` UNIQUE, `matricule` UNIQUE, NOT NULL sur champs obligatoires
- index sur `email` et `matricule`
- migration Flyway `V2__create_users.sql`
- données de test : 1 admin, 2 enseignants, 5 étudiants (seed SQL ou `CommandLineRunner`)

4. Diagrammes UML à modifier

- Diagramme de classes : ajouter `Utilisateur`, `Role`, `Enseignant`, `Etudiant`, `Administrateur` avec attributs privés/méthodes publiques (déjà fait — vérifier cohérence avec l'implémentation réelle)
- Use Case : ajouter les cas "S'inscrire", "Se connecter", "Gérer les comptes" (acteurs : Admin, Enseignant, Étudiant)
- Diagramme de séquence : séquence "Connexion utilisateur" (saisie → vérification → génération JWT → redirection selon rôle)

5. Base de données

- Nouvelles tables: `utilisateur`, `role`, `enseignant`, `etudiant`, `administrateur`
- Clé étrangère: `utilisateur.role_id → role.id`

- Contraintes : UNIQUE(email), UNIQUE(matricule), CHECK sur format email si applicable
- Migration Flyway : `V2__create_users.sql`
- Données de test : jeu de comptes de démonstration pour chaque rôle

6. API à développer

POST /auth/register

- Body : `{ email, motDePasse, nom, prenom, role }`
- Réponse : `201 Created` + `UserResponseDTO`
- Erreurs : `409` email déjà utilisé, `400` validation échouée

POST /auth/login

- Body : `{ email, motDePasse }`
- Réponse : `200 OK` + `{ token, utilisateur }`
- Erreurs : `401` identifiants invalides, `403` compte désactivé

GET /utilisateurs/me

- Réponse : `200 OK` + profil de l'utilisateur connecté
- Erreurs : `401` non authentifié

PUT /utilisateurs/me

- Body : `UpdateProfileDTO`
- Réponse : `200 OK` + profil mis à jour
- Erreurs : `400` validation échouée

GET /utilisateurs (Admin)

- Paramètres : `?role=&actif=&page=&size=`
- Réponse : `200 OK` + liste paginée

PUT /utilisateurs/{id}/desactiver (Admin)

- Réponse : `200 OK`
- Erreurs : `404` utilisateur introuvable

7. Interface utilisateur

- **Wireframe logique** : Login → Dashboard (selon rôle) ; Register (si ouvert au public) ; page Profil ; page Admin > Liste utilisateurs
- Composants React : `LoginForm`, `RegisterForm`, `ProfileCard`, `UserTable`, `RoleBadge`, `ProtectedRoute`
- Organisation des pages : `/login`, `/register`, `/profile`, `/admin/users`
- Navigation : redirection automatique selon rôle après connexion
- Responsive : formulaires en colonne unique sur mobile
- Animations : transition douce d'apparition des messages d'erreur

8. Tests

- Tests unitaires : hash du mot de passe, génération/validation JWT, validations DTO
- Tests d'intégration : `AuthService`, `UtilisateurService` avec base H2/Testcontainers
- Tests fonctionnels : parcours complet inscription → connexion → édition profil
- Tests API : chaque endpoint (succès + erreurs)
- Cas limites : email en majuscules/minuscules, mot de passe vide, double inscription simultanée
- Gestion des erreurs : messages homogènes (format JSON d'erreur standard)

9. Critères d'acceptation

- Un utilisateur peut s'inscrire et se connecter avec un token valide
- Les routes protégées refusent l'accès sans token ou avec un rôle insuffisant
- Un compte désactivé ne peut plus se connecter
- Le profil est modifiable et les changements persistent en base

10. Dépendances

- Doit être terminé avant : tous les autres sprints (chaque module dépend de l'authentification et des rôles)
- Peut être développé en parallèle : diagrammes UML, maquettes Figma des écrans suivants
- Risques : mauvaise conception des rôles/permissions difficile à corriger après coup → valider la matrice des permissions avant de coder les autres modules

11. Livrables

- Authentification JWT fonctionnelle de bout en bout
- Gestion des rôles opérationnelle (Admin/Enseignant/Étudiant)
- CRUD profil utilisateur complet

12. Démonstration du sprint

- Créer un compte enseignant en tant qu'admin, se connecter avec, modifier son profil, montrer le blocage d'accès pour un compte désactivé

13. Check-list de validation

- ☐ Base de données créée (utilisateur, role, enseignant, etudiant, administrateur)
- ☐ API d'authentification fonctionnelle
- ☐ Tests OK (unitaires, intégration, API)
- ☐ Frontend connecté (login, register, profil)
- ☐ Documentation Swagger à jour
- ☐ UML mis à jour (classes, use case, séquence connexion)
- ☐ Git propre (PR reviewée, branche fusionnée)
- ☐ Déploiement validé sur l'environnement de dev

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Entités + héritage JPA	Moyen	1 j	Haute
Spring Security + JWT	Difficile	2 j	Haute
CRUD profil	Facile	1 j	Haute
Frontend login/register	Moyen	1.5 j	Haute
Frontend admin users	Moyen	1 j	Moyenne
Tests	Moyen	1.5 j	Haute

15. Bonnes pratiques

- Mots de passe jamais stockés en clair (BCrypt)
- JWT avec expiration courte + refresh token si possible
- Principe SOLID : séparer clairement `AuthService` et `UtilisateurService`
- Validation systématique côté backend (ne jamais faire confiance au frontend)
- Nommage cohérent des DTO (`XxxRequestDTO`) / (`XxxResponseDTO`)

SPRINT 2 — Structure Scolaire : Classes & Matières (2 semaines)

1. Objectif du sprint

Permettre aux enseignants et administrateurs de gérer les classes, les matières, et leurs associations (un enseignant enseigne une matière, une matière est dispensée dans des classes, une classe contient des étudiants).

2. User Stories

- En tant qu'administrateur, je veux créer des classes et des matières, afin de structurer l'établissement.
- En tant qu'administrateur, je veux affecter un enseignant à une matière, afin de définir qui peut créer des QCM sur cette matière.
- En tant qu'enseignant, je veux voir la liste de mes matières et classes, afin de préparer mes QCM en conséquence.
- En tant qu'étudiant, je veux être rattaché à une classe, afin d'accéder aux QCM qui me concernent.

3. Tâches détaillées

Backend

- créer le modèle `Classe`, `Matiere`

- créer le schéma des associations (Enseignant↔Matiere, Matiere↔Classe, Classe↔Etudiant)
- créer les validations (nom de classe unique par année scolaire, code matière unique)
- créer les services (`ClasseService`, `MatiereService`)
- créer les routes CRUD
- gérer les erreurs (matière déjà affectée, classe pleine si limite définie)
- écrire les tests
- documenter l'API

Frontend

- créer les pages "Gestion des classes" et "Gestion des matières" (admin)
- créer les composants (formulaire classe, formulaire matière, sélecteur enseignant/matière)
- créer les formulaires d'affectation (enseignant → matière, matière → classes)
- connecter l'API
- gérer les erreurs et le loading
- responsive design (tableaux adaptatifs sur mobile)

Base de données

- créer les tables (`classe`, `matiere`)
- table de jointure (`matiere_classe`) (dispenséeDans, relation N-N)
- relation (`enseignant_matiere`) (N-N) si un enseignant peut avoir plusieurs matières et une matière plusieurs enseignants — sinon FK simple selon le modèle validé
- clé étrangère (`etudiant.classe_id → classe.id`)
- contraintes : UNIQUE(code) sur matière
- index sur (`classe.nom`), (`matiere.code`)
- migration Flyway (`V3__create_structure_scolaire.sql`)
- données de test : 3 classes, 5 matières, affectations de démonstration

4. Diagrammes UML à modifier

- Diagramme de classes : intégrer (`Classe`), (`Matiere`) et les relations (déjà présentes dans le diagramme existant — vérifier les cardinalités par rapport à l'implémentation réelle des tables de jointure)
- Use Case : ajouter "Gérer les classes", "Gérer les matières", "Affecter un enseignant"
- Diagramme d'activité : processus "Affectation d'un enseignant à une matière et des classes"

5. Base de données

- Nouvelles tables : (`classe`), (`matiere`), (`matiere_classe`)
- Nouvelles relations : (`etudiant.classe_id`), (`matiere_classe (matiere_id, classe_id)`), (`enseignant_matiere`)

- Clés étrangères avec `ON DELETE RESTRICT` (ne pas supprimer une classe contenant des étudiants)
- Migration Flyway `v3__create_structure_scolaire.sql`
- Données de test représentatives

6. API à développer

POST /classes — Body: `{ nom, niveau, anneeScolaire }` → `201` / `400`

GET /classes — Paramètres: `?niveau=&anneeScolaire=` → `200` + liste

PUT /classes/{id} — Body: champs modifiables → `200` / `404`

DELETE /classes/{id} → `204` / `409` si étudiants rattachés

POST /matieres — Body: `{ code, nom, description, couleur }` → `201` / `400` (code dupliqué)

GET /matieres → `200` + liste

POST /matieres/{id}/classes — Body: `{ classeIds: [] }` → `200` (affectation)

POST /enseignants/{id}/matieres — Body: `{ matiereIds: [] }` → `200`

7. Interface utilisateur

- Wireframe : Admin > Classes (liste + création) ; Admin > Matières (liste + création) ; Admin > Affectations
- Composants: `ClasseTable`, `ClasseForm`, `MatiereTable`, `MatiereForm`, `AffectationSelector`
- Organisation: `/admin/classes`, `/admin/matieres`, `/admin/affectations`
- Navigation : onglets latéraux dans l'espace admin
- Responsive : cartes empilées sur mobile au lieu de tableaux larges

8. Tests

- Unitaires : validations (code matière unique, nom classe unique)
- Intégration : `ClasseService`, `MatiereService` avec relations N-N
- Fonctionnels : créer classe → créer matière → affecter enseignant → affecter classes
- API : tous les endpoints CRUD
- Cas limites : suppression d'une classe avec étudiants, affectation en double
- Gestion des erreurs : messages explicites (409 pour conflits de suppression)

9. Critères d'acceptation

- Une classe et une matière peuvent être créées, modifiées, supprimées (si vide)
- Un enseignant voit uniquement ses matières et classes affectées
- Les associations N-N sont correctement persistées et affichées

10. Dépendances

- Doit être terminé avant : Sprint 3 (un QCM est rattaché à une matière et créé par un enseignant affecté à celle-ci)
- Peut être développé en parallèle : maquettes Figma des écrans de création de QCM
- Risques : modèle de relation enseignant-matière-classe mal défini dès le départ → clarifier avec l'encadrant si un enseignant peut enseigner plusieurs matières

11. Livrables

- Gestion complète des classes et matières
- Affectations enseignant/matière/classe fonctionnelles

12. Démonstration du sprint

- Créer une classe, une matière, affecter un enseignant, montrer que l'enseignant voit bien sa matière dans son espace

13. Check-list de validation

- ☐ Tables classe/matière/jointures créées
- ☐ API CRUD fonctionnelle
- ☐ Tests OK
- ☐ Frontend admin connecté
- ☐ Documentation à jour
- ☐ UML mis à jour
- ☐ Git propre
- ☐ Déploiement validé

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Entités + relations N-N	Moyen	1.5 j	Haute
API CRUD classes/matières	Facile	1 j	Haute
API affectations	Moyen	1 j	Haute
Frontend admin	Moyen	1.5 j	Moyenne
Tests	Moyen	1 j	Haute

15. Bonnes pratiques

- Éviter les suppressions en cascade destructrices (`ON DELETE RESTRICT`) par défaut
 - DTOs dédiés pour les relations N-N (éviter d'exposer les entités JPA brutes → risque de boucles infinies en sérialisation)
 - Tests d'intégration sur les relations avant de passer au sprint suivant
-

SPRINT 3 — Création de QCM, Questions & Réponses (2 semaines)

1. Objectif du sprint

Permettre aux enseignants de créer des QCM complets avec questions (plusieurs types) et réponses, via une interface de type "builder".

2. User Stories

- En tant qu'enseignant, je veux créer un QCM avec titre, description et difficulté, afin de préparer une évaluation.
- En tant qu'enseignant, je veux ajouter des questions de différents types (choix unique, choix multiple, vrai/faux, image, média), afin de varier les évaluations.
- En tant qu'enseignant, je veux définir les réponses correctes et un feedback, afin de guider l'étudiant après sa réponse.
- En tant qu'enseignant, je veux réordonner mes questions, afin de structurer le déroulement du QCM.
- En tant qu'enseignant, je veux dupliquer/archiver un QCM, afin de le réutiliser ou le retirer sans le supprimer.

3. Tâches détaillées

Backend

- créer le modèle `QCM`, `Question`, `Reponse`
- créer les schémas/DTOs (`QCMCreateDTO`, `QuestionDTO`, `ReponseDTO`, `QCMDetailDTO`)
- créer les validations (au moins 1 question par QCM, au moins 2 réponses par question, au moins 1 réponse correcte)
- créer les services (`QCMService`, `QuestionService`) avec logique de création imbriquée (QCM + questions + réponses en une transaction)
- créer les routes (CRUD QCM, gestion questions/réponses, réordonnancement, duplication, archivage)
- gérer les erreurs (QCM sans question, question sans réponse correcte)
- écrire les tests
- documenter l'API

Frontend

- créer la page "Liste de mes QCM" (enseignant)
- créer la page "Builder de QCM" (formulaire multi-étapes ou drag & drop)
- créer les composants (`QuestionCard`, `ReponseInput` selon type, sélecteur de type de question, upload image/média)
- créer les formulaires dynamiques selon `TypeQuestionEnum`
- connecter l'API (création, sauvegarde auto/brouillon, publication)
- gérer les erreurs (validation en temps réel : question sans réponse correcte)

- gérer le loading (sauvegarde en cours)
- responsive design (builder utilisable sur tablette au minimum)

Base de données

- créer les tables `qcm`, `question`, `reponse`
- relations : `qcm.enseignant_id`, `qcm.matiere_id`, `question.qcm_id`, `reponse.question_id`
- contraintes : NOT NULL sur champs obligatoires, CHECK sur `ordre >= 0`
- index sur `qcm.matiere_id`, `question.qcm_id`
- migration Flyway `V4__create_qcm_questions.sql`
- données de test : 3 QCM complets avec questions variées

4. Diagrammes UML à modifier

- Diagramme de classes : vérifier la composition `QCM *-- Question *-- Reponse` (déjà modélisée) — s'assurer que l'implémentation respecte les cardinalités (1..* questions, 2..* réponses)
- Use Case : ajouter "Créer un QCM", "Ajouter une question", "Publier/Archiver un QCM"
- Diagramme de séquence : "Création d'un QCM complet" (enseignant → formulaire → validation → sauvegarde transactionnelle)
- Diagramme d'activité : workflow de création d'une question selon son type (branchement selon `TypeQuestionEnum`)

5. Base de données

- Nouvelles tables : `qcm`, `question`, `reponse`
- Nouvelles relations : `qcm.enseignant_id → enseignant.id`, `qcm.matiere_id → matiere.id`, `question.qcm_id → qcm.id` (cascade delete), `reponse.question_id → question.id` (cascade delete)
- Contraintes : CHECK au moins une réponse correcte (validée côté application, pas en SQL pur)
- Migration Flyway `V4__create_qcm_questions.sql`
- Données de test : QCM variés (facile/moyen/difficile, types de questions différents)

6. API à développer

POST /qcm — Body : `QCMCreateDTO` (titre, description, difficulté, matiereId, questions[]) → `201` / `400`

GET /qcm — Paramètres : `?matiereId=&difficulte=&archive=&page=` → `200` + liste paginée

GET /qcm/{id} → `200` + détail complet (questions + réponses) / `404`

PUT /qcm/{id} → `200` / `400` / `404`

DELETE /qcm/{id} → `204` / `409` si sessions déjà lancées dessus

POST /qcm/{id}/dupliquer → `201` + nouveau QCM

PUT /qcm/{id}/archiver → (200)

PUT /qcm/{id}/questions/reorder — Body: { ordre: [questionId...] } → (200)

7. Interface utilisateur

- Wireframe : Liste QCM (cartes avec filtres) → Builder (étapes : Infos générales → Questions → Aperçu) → Publication
- Composants : (QCMCard), (QCMBuilder), (QuestionEditor), (ReponseEditor), (TypeQuestionSelector), (MediaUploader)
- Organisation : (/enseignant/qcm), (/enseignant/qcm/nouveau), (/enseignant/qcm/{id}/editer)
- Navigation : bouton flottant "Ajouter une question", sauvegarde de brouillon automatique
- Responsive : builder simplifié sur mobile (édition d'une question à la fois)
- Animations : transition entre les étapes du builder

8. Tests

- Unitaires : validation "au moins une réponse correcte", calcul du score total
- Intégration : création transactionnelle QCM+questions+réponses (rollback si erreur)
- Fonctionnels : créer un QCM complet de bout en bout, dupliquer, archiver
- API : tous les endpoints + réordonnancement
- Cas limites : question sans réponse correcte, QCM sans question, upload de fichier trop volumineux
- Gestion des erreurs : rollback complet si une question est invalide

9. Critères d'acceptation

- Un enseignant peut créer un QCM complet avec au moins une question et récupérer son détail
- Chaque type de question s'affiche et se configure correctement dans le builder
- La duplication et l'archivage fonctionnent sans perte de données

10. Dépendances

- Doit être terminé avant : Sprint 4 (une session live lance un QCM existant)
- Peut être développé en parallèle : Sprint 5 (badges) n'a pas de dépendance directe
- Risques : gestion des uploads média (stockage) à définir tôt (local/S3/Cloudinary) pour éviter de refaire le module

11. Livrables

- Builder de QCM fonctionnel avec tous les types de questions
- API complète de gestion des QCM

12. Démonstration du sprint

- Créer un QCM de A à Z avec 3 types de questions différents, le dupliquer, l'archiver

13. Check-list de validation

- ☐ Tables qcm/question/reponse créées
- ☐ API QCM fonctionnelle (CRUD + duplication + archivage)
- ☐ Tests OK
- ☐ Builder frontend connecté
- ☐ Documentation à jour
- ☐ UML mis à jour (classes, use case, séquence, activité)
- ☐ Git propre
- ☐ Déploiement validé

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Entités QCM/Question/Reponse	Moyen	1 j	Haute
Service création transactionnelle	Difficile	1.5 j	Haute
Upload média	Moyen	1 j	Moyenne
Builder frontend (multi-étapes)	Difficile	2.5 j	Haute
Tests	Moyen	1.5 j	Haute

15. Bonnes pratiques

- Transactions atomiques pour la création imbriquée (rollback total en cas d'erreur)
- Validation métier centralisée dans le service, pas dans le contrôleur
- Limiter la taille/le type des fichiers uploadés (sécurité)
- Découpage du builder en petits composants réutilisables (Clean Code, SRP)

SPRINT 4 — Sessions Live & Participations (Kahoot-style) (2 semaines)

1. Objectif du sprint

Implémenter le cœur "Kahoot-like" : lancement d'une session en temps réel, rejoindre via code PIN, réponse en direct, classement live, y compris pour les invités sans compte.

2. User Stories

- En tant qu'enseignant, je veux lancer une session à partir d'un QCM, afin que mes étudiants puissent y participer en direct.
- En tant qu'étudiant (ou invité), je veux rejoindre une session avec un code PIN, afin de participer sans friction.
- En tant que participant, je veux répondre aux questions dans le temps imparti, afin d'obtenir un score.

- En tant qu'enseignant, je veux voir le classement en temps réel, afin de dynamiser la session.

3. Tâches détaillées

Backend

- créer le modèle (SessionQuiz), (Participation), (ReponseEtudiant), (StatistiqueQuestion)
- créer les DTOs (SessionCreateDTO, JoinSessionDTO, ReponseSubmitDTO, LeaderboardDTO)
- créer les validations (code PIN unique actif, session non pleine, non terminée)
- créer le service de session (génération code PIN, gestion du cycle de vie : PLANIFIEE → EN_COURS → TERMINEE)
- configurer WebSocket (STOMP) pour la diffusion temps réel (question suivante, classement, fin de session)
- créer la logique de calcul de score (rapidité + exactitude)
- gérer la participation invité ((etudiant) nullable, (pseudonyme) obligatoire)
- gérer les erreurs (PIN invalide, session déjà terminée, session pleine)
- écrire les tests
- documenter l'API

Frontend

- créer la page "Lancer une session" (enseignant, choix du QCM + mode)
- créer la page "Rejoindre une session" (saisie PIN + pseudonyme si invité)
- créer l'écran "Salle d'attente" (liste des participants en temps réel)
- créer l'écran "Question en cours" (timer, choix de réponse, feedback immédiat)
- créer l'écran "Classement" (leaderboard animé après chaque question)
- connecter le WebSocket (abonnement aux canaux de session)
- gérer les erreurs (déconnexion, PIN invalide)
- gérer le loading (attente de la question suivante)
- responsive design (écran participant optimisé mobile, écran enseignant optimisé grand écran/projecteur)

Base de données

- créer les tables (session_quiz), (participation), (reponse_etudiant), (statistique_question)
- relations: (session_quiz.qcm_id), (participation.session_id), (participation.etudiant_id) (nullable), (reponse_etudiant.participation_id), (reponse_etudiant.question_id)
- contraintes: UNIQUE((code_pin)) parmi les sessions actives
- index sur (session_quiz.code_pin), (participation.session_id)
- migration Flyway (V5__create_sessions.sql)
- données de test : sessions de démonstration (terminées et planifiées)

4. Diagrammes UML à modifier

- Diagramme de classes : vérifier `Participation` (etudiant `0..1`, note sur la gestion des invités déjà présente) et `ReponseEtudiant`
- Use Case : ajouter "Lancer une session", "Rejoindre une session (invité/étudiant)", "Répondre en direct"
- Diagramme de séquence : "Déroulement complet d'une session live" (lancement → join → question → réponse → score → classement → fin), incluant les échanges WebSocket
- Diagramme d'activité : cycle de vie d'une `SessionQuiz` (PLANIFIEE → EN_COURS → TERMINEE/ANNULEE)

5. Base de données

- Nouvelles tables : `session_quiz`, `participation`, `reponse_etudiant`, `statistique_question`
- Nouvelles relations : FK vers `qcm`, `etudiant` (nullable), `question`
- Contraintes : unicité du code PIN actif, `terminee` par défaut à `false`
- Migration Flyway `V5__create_sessions.sql`
- Données de test : historique de sessions pour tester les statistiques

6. API à développer

POST /sessions — Body : `{ qcmId, mode, nombreMaxParticipants, dateDebutPlanifiee }` → `201` + code PIN

POST /sessions/{pin}/rejoindre — Body : `{ etudiantId? , pseudonyme? }` → `200` + token de participation / `404` PIN invalide / `409` session pleine ou terminée

POST /sessions/{id}/demarrer (enseignant) → `200`

POST /participations/{id}/reponses — Body : `{ questionId, choixSelectionnes, tempsReponse }` → `200` + score obtenu / `400` question déjà répondue

GET /sessions/{id}/classement → `200` + leaderboard

PUT /sessions/{id}/terminer → `200`

WebSocket `/topic/session/{id}` : diffusion question courante, classement, fin de session

7. Interface utilisateur

- Wireframe : Enseignant (choix QCM → salle d'attente → contrôle du déroulement → classement final) ; Participant (join → salle d'attente → question → feedback → classement)
- Composants : `SessionLauncher`, `JoinSessionForm`, `WaitingRoom`, `QuestionLive`, `Timer`, `Leaderboard`, `ScoreFeedback`
- Organisation : `/enseignant/sessions/nouvelle`, `/session/{id}/host`, `/session/rejoindre`, `/session/{id}/play`
- Navigation : passage automatique d'écran piloté par les événements WebSocket

- Responsive : interface participant pensée mobile-first (gros boutons de réponse)
- Animations : compte à rebours du timer, animation du classement (montée/descente de rang)

8. Tests

- Unitaires : calcul du score (rapidité + exactitude), génération de code PIN unique
- Intégration : cycle de vie complet d'une session (création → join → réponses → fin)
- Fonctionnels : scénario multi-participants (dont un invité) sur une session simulée
- Tests API : tous les endpoints REST + connexion WebSocket
- Cas limites : deux participants répondent en même temps, participant qui se déconnecte en cours de session, session qui atteint le nombre max de participants
- Gestion des erreurs : reconnexion après perte de connexion WebSocket

9. Critères d'acceptation

- Une session peut être créée, rejointe (avec ou sans compte), et menée jusqu'au classement final
- Le score est calculé correctement selon rapidité et exactitude
- Le classement se met à jour en temps réel pour tous les participants connectés

10. Dépendances

- Doit être terminé avant : Sprint 5 (les badges et l'historique dépendent des participations terminées)
- Dépend de : Sprint 3 (un QCM doit exister pour lancer une session)
- Risques : la partie temps réel (WebSocket) est la plus technique du projet → prévoir une marge de temps supplémentaire, tester la montée en charge avec plusieurs participants simulés

11. Livrables

- Sessions live fonctionnelles de bout en bout (hôte + participants, y compris invités)
- Classement en temps réel opérationnel

12. Démonstration du sprint

- Lancer une session, faire rejoindre 2-3 participants (dont un invité), répondre aux questions en direct, montrer le classement final

13. Check-list de validation

- ☐ Tables sessions/participations créées
- ☐ API sessions + WebSocket fonctionnels
- ☐ Tests OK (y compris scénario multi-participants)
- ☐ Frontend hôte + participant connectés
- ☐ Documentation à jour
- ☐ UML mis à jour (classes, use case, séquence, activité)
- ☐ Git propre

- ☐ Déploiement validé

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Entités sessions/participations	Moyen	1 j	Haute
Configuration WebSocket (STOMP)	Difficile	2 j	Haute
Calcul de score + classement	Moyen	1 j	Haute
Frontend salle d'attente + jeu live	Difficile	2.5 j	Haute
Gestion invités	Facile	0.5 j	Moyenne
Tests (dont multi-participants)	Difficile	2 j	Haute

15. Bonnes pratiques

- Isoler la logique temps réel dans une couche dédiée (`SessionWebSocketController` séparé du `SessionService`)
- Idempotence sur la soumission de réponse (éviter le double comptage en cas de re-envoi réseau)
- Sécuriser les canaux WebSocket (vérifier que seul le participant/hôte légitime peut publier)
- Prévoir une stratégie de reconnexion côté frontend

SPRINT 5 — Gamification, Notifications, Historique & Export (2 semaines)

1. Objectif du sprint

Ajouter la couche d'engagement (badges), la communication (notifications), la traçabilité (historique des actions) et les exports de résultats.

2. User Stories

- En tant qu'étudiant, je veux gagner des badges selon mes performances, afin d'être motivé à progresser.
- En tant qu'utilisateur, je veux recevoir des notifications (email/in-app), afin d'être informé des événements importants.
- En tant qu'administrateur, je veux consulter l'historique des actions, afin de suivre l'activité de la plateforme.
- En tant qu'enseignant, je veux exporter les résultats d'une session en PDF/Excel/CSV, afin de les partager ou archiver.

3. Tâches détaillées

Backend

- créer le modèle `Badge`, `BadgeEtudiant`, `Notification`, `HistoriqueAction`, `Export`
- créer les schémas/DTOs correspondants
- créer les validations (condition d'attribution de badge, format d'export valide)
- créer les services : `BadgeService` (règles d'attribution automatique), `NotificationService` (envoi email via SMTP + in-app), `HistoriqueService` (log automatique via AOP/interceptor), `ExportService` (génération PDF/Excel/CSV)
- créer les routes correspondantes
- gérer les erreurs (échec d'envoi email, génération d'export)
- écrire les tests
- documenter l'API

Frontend

- créer la page "Mes badges" (étudiant)
- créer le centre de notifications (icône cloche + liste, marquer comme lu)
- créer la page "Historique" (admin, filtrable par utilisateur/action/date)
- créer le bouton "Exporter" sur les résultats de session (choix du format)
- connecter l'API
- gérer les erreurs et le loading
- responsive design

Base de données

- créer les tables `badge`, `badge_etudiant`, `notification`, `historique_action`, `export`
- relations : `badge_etudiant.badge_id`, `badge_etudiant.etudiant_id`, `notification.utilisateur_id`, `historique_action.utilisateur_id`, `export.enseignant_id`, `export.session_id` (nullable)
- contraintes : NOT NULL sur champs obligatoires
- index sur `notification.utilisateur_id`, `historique_action.date`
- migration Flyway `V6__create_gamification_notif_export.sql`
- données de test : quelques badges prédéfinis (ex : "Premier Quiz", "Sans Faute", "Rapide comme l'éclair")

4. Diagrammes UML à modifier

- Diagramme de classes : vérifier `Badge`, `BadgeEtudiant`, `Notification`, `HistoriqueAction`, `Export` (déjà présents)
- Use Case : ajouter "Consulter mes badges", "Recevoir une notification", "Exporter les résultats", "Consulter l'historique"
- Diagramme de séquence : "Attribution automatique d'un badge après une participation" ; "Génération d'un export PDF"
- Diagramme d'activité : règles de calcul d'attribution des badges (conditions multiples)

5. Base de données

- Nouvelles tables : `badge`, `badge_etudiant`, `notification`, `historique_action`, `export`

- Nouvelles relations : FK vers `etudiant`, `utilisateur`, `enseignant`, `session_quiz`
- Contraintes : unicité (un même badge ne peut être attribué deux fois pour la même condition, selon règle métier)
- Migration Flyway `V6__create_gamification_notif_export.sql`
- Données de test : badges de base + notifications de démonstration

6. API à développer

GET `/etudiants/{id}/badges` → `200` + liste des badges obtenus

GET `/notifications` (utilisateur connecté) → `200` + liste

PUT `/notifications/{id}/lu` → `200`

GET `/historique` (admin) — Paramètres : `{ ? utilisateurId=&typeAction=&dateDebut=&dateFin= }` → `200` + liste paginée

POST `/sessions/{id}/export` — Body : `{ type: PDF | EXCEL | CSV }` → `200` + lien de téléchargement / `400` type invalide

7. Interface utilisateur

- Wireframe : page Badges (grille de badges obtenus/verrouillés) ; centre de notifications (dropdown) ; page Historique (tableau filtrable) ; bouton export sur la page résultats de session
- Composants : `BadgeGrid`, `BadgeCard`, `NotificationBell`, `NotificationList`, `HistoriqueTable`, `ExportButton`
- Organisation : `/etudiant/badges`, composant notifications global dans le header, `/admin/historique`
- Navigation : accès rapide aux notifications depuis n'importe quelle page
- Responsive : grille de badges qui passe de 4 à 2 colonnes sur mobile
- Animations : animation de déblocage de badge (confetti/pop-up léger)

8. Tests

- Unitaires : règles d'attribution de badge, formatage des exports
- Intégration : `BadgeService` déclenché après une participation terminée
- Fonctionnels : terminer une session → vérifier badge attribué + notification reçue
- Tests API : tous les endpoints
- Cas limites : badge déjà attribué (pas de doublon), export d'une session sans participants
- Gestion des erreurs : échec d'envoi email n'empêche pas le reste du flux (traitement asynchrone)

9. Critères d'acceptation

- Les badges sont attribués automatiquement selon les règles définies après chaque participation

- Les notifications s'affichent en temps réel (ou au rechargement) et peuvent être marquées comme lues
- L'export génère un fichier valide dans les 3 formats proposés
- L'historique est consultable et filtrable par un administrateur

10. Dépendances

- Doit être terminé avant : Sprint 7 (finalisation) uniquement
- Dépend de : Sprint 4 (les badges/exports se basent sur les participations et sessions terminées)
- Peut être développé en parallèle : Sprint 6 (Module IA) — aucune dépendance directe
- Risques : l'envoi d'email peut être lent/bloquant → traiter en asynchrone (ex : `@Async` Spring ou file d'attente)

11. Livrables

- Système de badges automatique fonctionnel
- Notifications email + in-app opérationnelles
- Historique consultable par l'admin
- Export PDF/Excel/CSV fonctionnel

12. Démonstration du sprint

- Terminer une session, montrer le badge débloqué, la notification reçue, puis exporter les résultats en PDF

13. Check-list de validation

- ☐ Tables badges/notifications/historique/export créées
- ☐ API fonctionnelle pour les 4 modules
- ☐ Tests OK
- ☐ Frontend connecté (badges, notifications, historique, export)
- ☐ Documentation à jour
- ☐ UML mis à jour
- ☐ Git propre
- ☐ Déploiement validé

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Entités + services badges	Moyen	1.5 j	Haute
Service notifications (email async)	Moyen	1 j	Moyenne
Service historique (log automatique)	Moyen	1 j	Moyenne
Service export PDF/Excel/CSV	Difficile	1.5 j	Moyenne
Frontend (badges, notifs, historique, export)	Moyen	2 j	Moyenne
Tests	Moyen	1.5 j	Haute

15. Bonnes pratiques

- Traitement asynchrone pour les emails (ne pas bloquer la requête principale)
- Historique généré automatiquement via un intercepteur/aspect (éviter de dupliquer le code de log partout)
- Génération d'export dans un service dédié, réutilisable pour d'autres modules futurs
- Nettoyage périodique des vieilles notifications (politique de rétention à définir)

SPRINT 6 — Module IA : Génération de QCM depuis des Documents (2 semaines)

1. Objectif du sprint

Permettre à un enseignant d'uploader un document (PDF/Word/PPT) et de générer automatiquement un QCM à partir de son contenu, via un pipeline d'extraction + génération IA suivi asynchrone.

2. User Stories

- En tant qu'enseignant, je veux uploader un document, afin de générer un QCM automatiquement à partir de son contenu.
- En tant qu'enseignant, je veux suivre l'état de la génération (en attente/en cours/terminé/échec), afin de savoir quand le QCM est prêt.
- En tant qu'enseignant, je veux relire et modifier le QCM généré avant publication, afin de garantir sa qualité.

3. Tâches détaillées

IA / Backend

- créer le modèle `DocumentSource`, `QuizGenerationJob`
- créer les DTOs (`DocumentUploadDTO`, `JobStatusDTO`)
- créer les validations (formats acceptés PDF/DOCX/PPTX, taille max)
- créer le service d'extraction de texte selon le type de document

- préparation des données (nettoyage du texte extrait, découpage en sections pertinentes)
- intégration IA : appel à un LLM (prompt de génération de questions à partir du texte, format de sortie structuré JSON)
- API IA : endpoint interne/service dédié pour la génération (peut être un microservice séparé)
- créer le service de job asynchrone (`QuizGenerationJob`) avec statuts, traitement en file d'attente ou `@Async`)
- optimisation : limiter la taille du texte envoyé au modèle (chunking), mise en cache des résultats intermédiaires
- créer les routes (upload, suivi de statut, récupération du QCM généré)
- gérer les erreurs (document illisible, échec de génération, timeout IA)
- écrire les tests
- documenter l'API

Frontend

- créer la page "Générer un QCM depuis un document"
- créer le composant d'upload avec drag & drop et barre de progression
- créer l'écran de suivi de génération (statut en temps réel : polling ou WebSocket)
- créer l'écran de relecture/édition du QCM généré avant validation (réutilise le builder du Sprint 3)
- connecter l'API
- gérer les erreurs (upload échoué, génération échouée avec message clair)
- gérer le loading (indicateur de progression du job)
- responsive design

Base de données

- créer les tables `document_source`, `quiz_generation_job`
- relations : `document_source.enseignant_id`, `quiz_generation_job.document_source_id`, `quiz_generation_job.enseignant_id`, `quiz_generation_job.qcm_id` (nullable jusqu'à génération réussie)
- contraintes : NOT NULL sur champs obligatoires
- index sur `quiz_generation_job.status`
- migration Flyway `V7__create_module_ia.sql`
- données de test : un job terminé et un job en échec pour tester l'affichage des deux cas

4. Diagrammes UML à modifier

- Diagramme de classes : vérifier `DocumentSource`, `QuizGenerationJob` et leur lien vers `QCM` (déjà modélisés)
- Use Case : ajouter "Uploader un document", "Générer un QCM par IA", "Suivre l'état d'une génération"

- Diagramme de séquence : "Pipeline complet de génération IA" (upload → extraction → appel LLM → parsing → création QCM → notification enseignant)
- Diagramme d'activité : cycle de vie d'un `QuizGenerationJob` (EN_ATTENTE → EN_COURS → TERMINE/ECHEC)
- Diagramme de composants : ajouter le service IA comme composant séparé s'il est externalisé
- Diagramme de déploiement : mettre à jour si le service IA tourne sur une infrastructure distincte

5. Base de données

- Nouvelles tables : `document_source`, `quiz_generation_job`
- Nouvelles relations : FK vers `enseignant`, `qcm` (nullable)
- Contraintes : `status` avec valeurs limitées à l'enum `StatusJobEnum`
- Migration Flyway `V7__create_module_ia.sql`
- Données de test : jobs à différents statuts

6. API à développer

POST /documents/upload — Body : `multipart/form-data` (fichier) → `201` + `DocumentSource` / `400` format non supporté / `413` fichier trop volumineux

POST /documents/{id}/generer-qcm — Body : `{ matiereId, nombreQuestions, difficulte }` → `202 Accepted` + `jobId`

GET /jobs/{id} → `200` + `{ status, logs, qcmId? }`

GET /jobs/{id}/qcm → `200` + QCM généré (prêt pour édition) / `409` si job non terminé

7. Interface utilisateur

- Wireframe : Upload document → Configuration génération (nb questions, difficulté) → Suivi (spinner/étapes) → Relecture QCM généré → Publication
- Composants : `DocumentUploader`, `GenerationConfigForm`, `JobStatusTracker`, `GeneratedQCMReview`
- Organisation : `/enseignant/ia/generer`, `/enseignant/ia/jobs/{id}`
- Navigation : notification/redirection automatique vers la relecture une fois le job terminé
- Responsive : upload utilisable au moins en tablette/desktop (usage enseignant principalement)
- Animations : barre de progression animée pendant l'upload et la génération

8. Tests

- Unitaires : extraction de texte par type de fichier, validation du format/de la taille
- Intégration : pipeline complet upload → job → QCM généré (avec IA mockée pour les tests)

- Fonctionnels : uploader un PDF de démonstration et vérifier qu'un QCM cohérent est généré
- Tests API : tous les endpoints, y compris le suivi de statut
- Cas limites : document vide/illisible, timeout de l'IA, document trop volumineux
- Gestion des erreurs : job marqué **ECHEC** avec logs exploitables, pas de blocage du reste de l'application

9. Critères d'acceptation

- Un enseignant peut uploader un document et suivre l'avancement de la génération jusqu'à son terme
- Le QCM généré est éditable avant publication (réutilisation du builder existant)
- Un échec de génération est clairement signalé avec un message compréhensible

10. Dépendances

- Dépend de : Sprint 3 (le QCM généré doit s'intégrer au modèle QCM/Question/Reponse existant)
- Peut être développé en parallèle : Sprint 5 (aucune dépendance directe)
- Risques : coût et latence des appels IA, qualité variable de la génération selon le document → prévoir une marge de test importante et un mécanisme de relance manuelle

11. Livrables

- Pipeline complet d'upload et de génération IA fonctionnel
- Intégration avec le builder de QCM existant pour la relecture

12. Démonstration du sprint

- Uploader un document de cours, lancer la génération, suivre le job jusqu'à son terme, relire et publier le QCM généré

13. Check-list de validation

- ☐ Tables document_source/quiz_generation_job créées
- ☐ API upload + génération + suivi fonctionnelle
- ☐ Tests OK (y compris cas d'échec)
- ☐ Frontend connecté (upload, suivi, relecture)
- ☐ Documentation à jour
- ☐ UML mis à jour (classes, use case, séquence, activité, composants/déploiement si applicable)
- ☐ Git propre
- ☐ Déploiement validé

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Entités + upload sécurisé	Moyen	1 j	Haute
Extraction de texte (PDF/DOCX/PPTX)	Difficile	1.5 j	Haute
Intégration LLM + parsing structuré	Difficile	2 j	Haute
Gestion asynchrone des jobs	Difficile	1.5 j	Haute
Frontend upload + suivi + relecture	Moyen	2 j	Haute
Tests (avec IA mockée)	Moyen	1.5 j	Haute

15. Bonnes pratiques

- Traitement asynchrone obligatoire (ne jamais bloquer une requête HTTP sur un appel IA long)
- Validation stricte des fichiers uploadés (type MIME réel, pas seulement l'extension) — sécurité
- Isoler l'appel au LLM dans un service dédié (facilite le changement de fournisseur plus tard)
- Toujours permettre une relecture humaine avant publication d'un contenu généré par IA
- Logs détaillés sur chaque étape du pipeline pour le débogage

SPRINT 7 — Finalisation, Sécurité, Optimisation & Déploiement (2 semaines)

1. Objectif du sprint

Consolider l'application : tests globaux, durcissement sécurité, optimisation des performances, documentation complète, préparation du déploiement final et de la soutenance.

2. User Stories

- En tant qu'utilisateur, je veux une application rapide et stable, afin d'avoir une bonne expérience même en cas de forte charge (session live à 30+ participants).
- En tant que responsable du projet, je veux une documentation complète, afin de pouvoir présenter et maintenir le projet.
- En tant qu'administrateur système, je veux un déploiement fiable et sécurisé, afin de mettre l'application en production.

3. Tâches détaillées

Backend

- audit de sécurité (injections, XSS, CSRF, gestion des rôles sur toutes les routes)
- revue des exceptions et des codes de retour HTTP sur l'ensemble de l'API
- optimisation des requêtes JPA (N+1, pagination, index manquants)
- mise en cache si pertinent (ex : liste des matières/classes peu volatiles)
- finalisation de la documentation Swagger/OpenAPI
- tests de charge sur le module sessions live (WebSocket)

Frontend

- audit d'accessibilité et de responsive design sur l'ensemble des écrans
- optimisation des performances (lazy loading des pages, compression des images)
- gestion globale des erreurs (page 404, page erreur serveur, boundary React)
- polish des animations et de la cohérence visuelle
- tests cross-browser de base

Base de données

- revue finale des contraintes, index, et clés étrangères sur toutes les tables
- script de données de démonstration complet pour la soutenance
- vérification de toutes les migrations Flyway (rejouabilité depuis zéro)

IA

- optimisation des prompts de génération (qualité/coût)
- gestion des cas limites du module IA (documents multi-langues, très longs documents)

Documentation & Démo

- rédaction du rapport technique / mémoire (architecture, choix techniques, diagrammes UML finaux)
- préparation du support de soutenance
- guide d'installation et de déploiement (README complet)

4. Diagrammes UML à modifier

- Mise à jour finale de **tous** les diagrammes (classes, use case, séquence, activité, composants, déploiement) pour refléter l'état réel du code livré
- Vérification de cohérence entre le diagramme de classes final et les entités JPA effectivement implémentées

5. Base de données

- Pas de nouvelle table — révision et durcissement du schéma existant
- Vérification des index de performance sur les tables les plus sollicitées (`session_quiz`, `participation`, `reponse_etudiant`)
- Jeu de données de démonstration finalisé pour la soutenance

6. API à développer

- Pas de nouvel endpoint métier — uniquement finalisation/durcissement des endpoints existants (validations, codes d'erreur homogènes, pagination généralisée)

7. Interface utilisateur

- Écran d'erreur générique (404 / 500)
- Polish visuel global (cohérence des couleurs, espacements, typographies sur toutes les pages)
- Vérification finale du responsive sur mobile/tablette/desktop pour tous les modules

8. Tests

- Tests unitaires : couverture de code globale (viser un seuil défini avec l'encadrant, ex : 70-80%)
- Tests d'intégration : parcours complets de bout en bout de chaque module
- Tests fonctionnels : scénario complet "enseignant crée un QCM, lance une session, étudiants participent, badges attribués, export généré"
- Tests API : suite de tests de non-régression complète
- Tests de charge : simulation de 30+ participants simultanés sur une session live
- Cas limites : revue globale de tous les cas limites identifiés dans les sprints précédents
- Gestion des erreurs : vérification de la cohérence des messages d'erreur sur toute l'application

9. Critères d'acceptation

- Aucune faille de sécurité critique identifiée (contrôle d'accès par rôle vérifié sur toutes les routes)
- L'application supporte une session live avec un nombre réaliste de participants sans dégradation majeure
- La documentation (technique + utilisateur) est complète et à jour
- Le déploiement fonctionne de bout en bout sur l'environnement cible

10. Dépendances

- Dépend de : tous les sprints précédents (1 à 6) doivent être fonctionnellement terminés
- Risques : sous-estimer le temps nécessaire à la correction des bugs découverts lors des tests de charge/sécurité → garder une marge de sécurité dans le planning global

11. Livrables

- Application complète, testée, sécurisée et déployée
- Documentation technique et utilisateur complète
- Diagrammes UML finaux cohérents avec le code
- Support de soutenance

12. Démonstration du sprint

- Démonstration complète de bout en bout de l'application devant le jury/encadrant : parcours enseignant + parcours étudiant + module IA + statistiques/export

13. Check-list de validation

- ☐ Audit de sécurité effectué et corrections appliquées
- ☐ Tests de charge validés sur les sessions live
- ☐ Couverture de tests satisfaisante
- ☐ Documentation technique complète
- ☐ Documentation utilisateur/guide d'installation complète
- ☐ Tous les diagrammes UML à jour et cohérents avec le code
- ☐ Git propre (historique clair, tags de version)
- ☐ Déploiement final validé en environnement de production/démo

14. Estimation

Tâche	Difficulté	Temps estimé	Priorité
Audit sécurité	Difficile	1.5 j	Haute
Optimisation performances backend	Moyen	1 j	Haute
Tests de charge sessions live	Difficile	1.5 j	Haute
Polish frontend	Moyen	1.5 j	Moyenne
Documentation technique/soutenance	Moyen	2 j	Haute
Déploiement final	Moyen	1 j	Haute

15. Bonnes pratiques

- Revue de code complète avant la version finale (Clean Code, SOLID, suppression du code mort)
 - Vérification systématique du principe du moindre privilège sur les rôles/permissions
 - Versionnement clair (tag Git de la version livrée, changelog)
 - Documentation vivante : garder les diagrammes UML synchronisés avec le code jusqu'au dernier commit
-

TABLEAU RÉCAPITULATIF GLOBAL

Sprint	Durée	Objectif	Fonctionnalités clés	Livrables	Difficulté globale	Priorité	Dépendances
Sprint 0	1 sem.	Setup technique	Squelette backend/frontend, CI/CD, BDD connectée	Environnement de dev fonctionnel	Facile	Haute	Aucune
Sprint 1	2 sem.	Auth & Utilisateurs	Inscription, connexion, rôles, profils	Authentification JWT complète	Difficile	Haute	Sprint 0
Sprint 2	2 sem.	Structure scolaire	Classes, matières, affectations	Gestion classes/matières opérationnelle	Moyen	Haute	Sprint 1
Sprint 3	2 sem.	QCM & Questions	Builder de QCM, types de questions variés	Création de QCM complète	Difficile	Haute	Sprint 2
Sprint 4	2 sem.	Sessions live	Code PIN, temps réel, classement	Sessions live fonctionnelles (Kahoot-like)	Difficile	Haute	Sprint 3
Sprint 5	2 sem.	Gamification & communication	Badges, notifications, historique, export	Engagement + traçabilité + export	Moyen	Moyenne	Sprint 4
Sprint 6	2 sem.	Module IA	Upload document, génération QCM par IA	Pipeline IA fonctionnel	Difficile	Moyenne	Sprint 3 (parallèle Sprint 5)
Sprint 7	2 sem.	Finalisation	Sécurité, performance, documentation, déploiement	Application livrée et soutenable	Difficile	Haute	Sprints 1 à 6

Durée totale estimée : ~17 semaines (≈ 4 mois), adaptable selon le rythme de travail (solo ou en binôme) et les retours de l'encadrant à chaque fin de sprint.